Ex 2.2.3: the multi-op grammar, with *red* anchored again

> The grammar grows from one operation to six: `mix` (`+`), `add`, `screen`, `multiply`, `lighten`, `darken`. We retrain on the new grammar to check the recipes: the un-anchored control, the ex-2.1.10 recipe, and three proposals from the ex-2.1.11 survey. The recipe carried over as it was. The proposals place *red* with a wider margin, and the frozen rule adopted one of them, but they put the axis on the syntax embeddings, so projecting it out breaks most non-red lines too; a post hoc read with the gates the rule left out narrows the choice to the recipe, and D2.2 builds on its short arm, which grades better for half the compute. Removal reads as partial on the saturating ops because their answers mostly do not depend on how red the red operand is.

Findings

- Task cost on the new grammar (H1) — **holds.** All 30 condition-op comparisons are within 0.02 of the control of the same length; the largest gap is -0.0094, `t00` on `mix`.
- Placement under the recipe reproduces (H2) — **holds.** Every placement criterion holds on the `mix` lines at five fresh seeds: seed-mean m_line 0.402, 0.96 of its ex-2.1.10 value, grading r^2 0.819, contrast 0.86, containment 0.062, no latched run.
- The plateau from the survey transfers (H3) — **holds.** A feasible proposal exceeds `recipe-short` (0.401) by more than a band (0.011): `t00` +0.075, `t48` +0.062, `t12` +0.059. Adopted: `t00`. The frozen rule adopts **`t00`**; a post hoc amendment that adds H2's lead gate and H4's selectivity gate, with the tie between the recipe lengths broken at twenty seeds (E6), adopts **`recipe`**. D2.2 builds on **`recipe-short`** by decision: it grades better and leads better at the embedding, with the same H4 read, for half the compute.
- Suppression transfers (H4) — **contrary.** `recipe`: red accuracy under projection 0.06–0.32 across the ops, `mix` non-red deficit 0.024; `t00`: red accuracy under projection 0.00–0.06 across the ops, `mix` non-red deficit 0.602. Removal misses on `add, screen, lighten, darken` for `recipe`. The `mix` deficit on `t00` is above 0.1.

> **How to read this draft**
>
> The hypotheses, the gates, the selection rule, and the method were frozen at commit `5a3233b`, before any anchored run, with the calibration recorded under the method. Each hypothesis section opens with its frozen prediction; its results follow in place. Anything we thought of after seeing the data is under Exploratory analyses, marked as post hoc. The op table and every count in the method are computed from `experiment.py` at render time.

Why this experiment

The claims of D2.2 are about an operation, and the D2.1 grammar has only one: `mix`, spelled `+`. The D2.2 plan makes the operation a variable. But every number D2.1 produced was measured on the one-op grammar: the recipe in ex-2.1.10, the operating-point plateau in the ex-2.1.11 survey, and the suppression figures in ex-2.2.1. The risk table in the plan names this ("recipe is grammar-specific") and sends it here.

So this is a regression check with a decision attached. We measure the same statistics on the same `mix` lines, and we freeze a rule that turns the proposals from the survey into the operating point the anchored-op experiments adopt.

Conditions

Every condition is trained here at fresh seeds. All but the richer-op arms use the six-op corpus at D2.1's size and epoch count, so the recipe's step count is unchanged.

condition	what	anchor	ops	lines per op	epochs (steps)	seeds
control	un-anchored	none	all six	16,666	100 (3,300)	5
control-short	un-anchored, at the proposals' length	none	all six	16,666	50 (1,650)	5
recipe	the ex-2.1.10 recipe	λ_a = 0.100 (annealed), τ = 0.100, anti 2.50 → 0.30 by 90%	all six	16,666	100 (3,300)	5
recipe-short	the recipe at the proposals' length	λ_a = 0.100 (annealed), τ = 0.100, anti 2.50 → 0.30 by 90%	all six	16,666	50 (1,650)	5
t00	survey proposal t00	λ_a = 0.557 (flat), τ = 0.265, anti 1.38 → 0.30 by 54%	all six	16,666	50 (1,650)	5
t48	survey proposal t48	λ_a = 0.431 (flat), τ = 0.222, anti 2.95 → 0.30 by 55%	all six	16,666	50 (1,650)	5
t12	survey proposal t12	λ_a = 0.361 (flat), τ = 0.233, anti 1.13 → 0.30 by 92%	all six	16,666	50 (1,650)	5
ops-1-corpus	un-anchored, mix only, D2.1's corpus size	none	`mix`	100,000	100 (3,300)	3
ops-3-corpus	un-anchored, mix, add, multiply, D2.1's corpus size	none	`mix`, `add`, `multiply`	33,333	100 (3,300)	3
ops-1-per-op	un-anchored, mix only, six-op lines per op	none	`mix`	16,666	550 (3,300)	3
ops-3-per-op	un-anchored, mix, add, multiply, six-op lines per op	none	`mix`, `add`, `multiply`	16,666	194 (3,298)	3
recipe-more	the recipe, addendum seeds	λ_a = 0.100 (annealed), τ = 0.100, anti 2.50 → 0.30 by 90%	all six	16,666	100 (3,300)	15
recipe-short-more	the recipe at 50 epochs, addendum seeds	λ_a = 0.100 (annealed), τ = 0.100, anti 2.50 → 0.30 by 90%	all six	16,666	50 (1,650)	15

The recipe is the D2.1 primary, `either-t100`. The only thing we changed is the grammar. The short arm of the recipe runs for as long as the proposals do, so H3 compares runs of equal length; the control has a short arm so that H1 can do the same.

The three proposals are the operating point from the survey plus the two promoted trials the survey read as its twin and its knee. `t48` sits within one m_line band of `t00`, with more grading and contrast. `t12` holds the most grading of the promoted set, at a margin the survey could not tell apart from the one for `t00`.

The richer-op arms are un-anchored models on one and three ops at the recipe's step count, in two matchings. The `corpus` arms keep D2.1's line count, and so have more lines per op. The `per-op` arms keep the six-op lines per op, and so have a smaller corpus, repeated over more epochs. Corpus size, lines per op, and step count cannot all be held while the op set changes, and the two matchings put the remaining difference on opposite sides, so the pair lets E4 read in both directions. They serve E4 only.

The interventions

We score the checkpoints of every candidate through the eval contract in `sca.intervention`, on the probe lines of each op.

- **`projection`** — the primary from ex-2.2.1: the axis projected out at every slice and position, at full strength. Read by H4.
- **Ride-along rows** — `operands` (the projection at the two operand positions only), `shaped` (the shaped suppression from M1, a = 0.5, b = 1, p = 1), and `ablate` (the axis weights zeroed). Reported without gates, so the intervention-tuning pass the plan schedules has the multi-op figures beside the one-op figures from ex-2.2.1.

Task cost on the new grammar (H1)

H1. Anchoring costs nothing on the task, on any op. We score the recipe at both lengths and the three proposals against the control of the same length, op by op, so the gate reads 30 condition-op comparisons. In each, seed-mean holdout exact-match accuracy is within 0.02 of its control's. Partial: every comparison within 0.05, or all but one within 0.02 and that one within 0.05. Contrary: a condition more than 0.05 below its control on some op, which would say that the anchor and reading the op compete with each other. A condition that far *above* its control on some op misses the gate too, and we would report it as an anomaly, since nothing in the design predicts one.

This gate only means something if the controls learn the grammar in the first place. We check that at both lengths in the calibration runs described in the method, before the freeze, rather than gating it here.

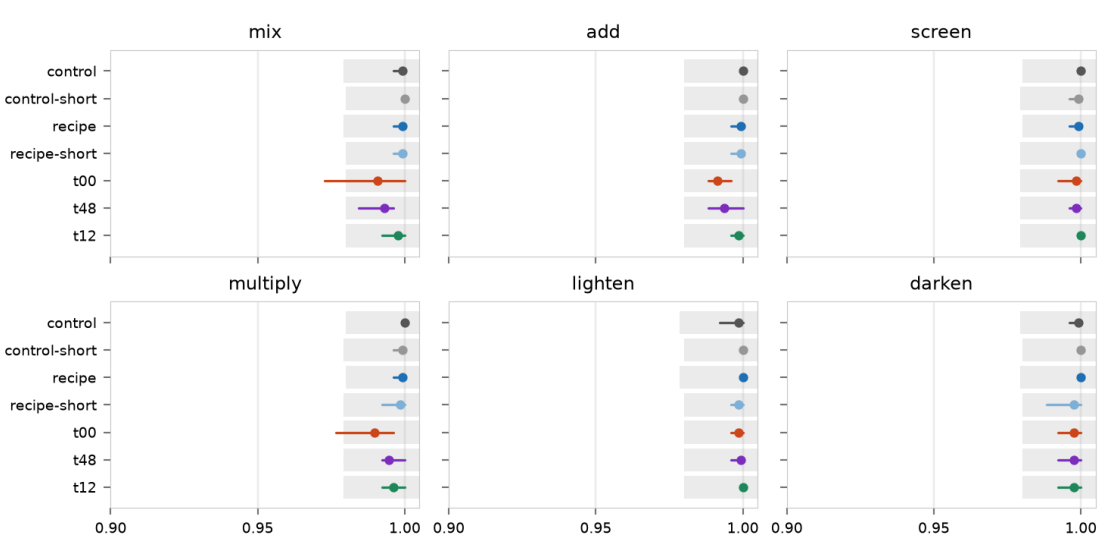
If the recipe misses this gate at full length, H2 goes unscored and the refuted H1 is the finding, as ex-2.1.10 ruled for its primary. H3 still runs, since its selection rule already requires the task gate of every candidate, and H4 is then scored on the adopted point alone. If no candidate is feasible, H4 goes unscored too.

condition	mix EM ↑	add EM ↑	screen EM ↑	multiply EM ↑	lighten EM ↑	darken EM ↑
control	0.999 ±0.002	1.000 ±0.000	1.000 ±0.000	1.000 ±0.000	0.998 ±0.004	0.999 ±0.002
control-short	1.000 ±0.000	1.000 ±0.000	0.999 ±0.002	0.999 ±0.002	1.000 ±0.000	1.000 ±0.000
recipe	0.999 ±0.002	0.999 ±0.002	0.999 ±0.002	0.999 ±0.002	1.000 ±0.000	1.000 ±0.000
recipe-short	0.999 ±0.002	0.999 ±0.002	1.000 ±0.000	0.998 ±0.004	0.998 ±0.002	0.998 ±0.006
t00	0.991 ±0.014	0.991 ±0.004	0.998 ±0.004	0.990 ±0.010	0.998 ±0.002	0.998 ±0.004
t48	0.993 ±0.006	0.994 ±0.006	0.998 ±0.002	0.995 ±0.004	0.999 ±0.002	0.998 ±0.004
t12	0.998 ±0.004	0.998 ±0.002	1.000 ±0.000	0.996 ±0.004	1.000 ±0.000	0.998 ±0.004
ops-1-corpus	0.999 ±0.002	·	·	·	·	·
ops-3-corpus	1.000 ±0.000	0.997 ±0.002	·	1.000 ±0.000	·	·
ops-1-per-op	0.917 ±0.014	·	·	·	·	·
ops-3-per-op	0.990 ±0.008	0.991 ±0.006	·	0.987 ±0.006	·	·
recipe-more	0.999 ±0.002	1.000 ±0.002	0.999 ±0.002	0.999 ±0.002	1.000 ±0.000	1.000 ±0.002
recipe-short-more	0.999 ±0.002	0.999 ±0.002	1.000 ±0.000	1.000 ±0.002	1.000 ±0.002	0.999 ±0.004

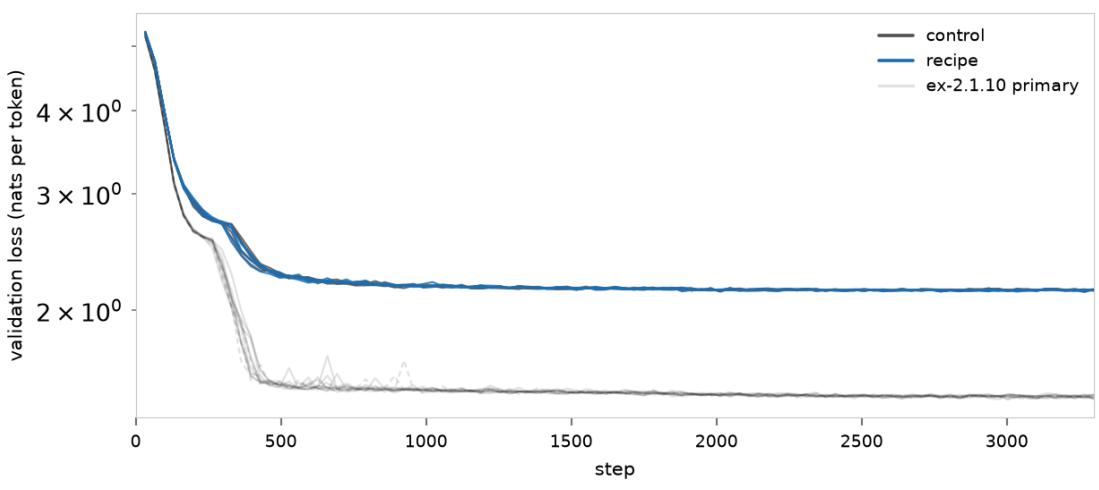
Holdout exact match per op, by condition: the seed mean with half the seed range beside it. The greyed rows are the two controls (100 and 50 epochs) and the richer-op arms, which carry no gate; a dot means the arm never trained on that op.

condition	control	mix gap	add gap	screen gap	multiply gap	lighten gap	darken gap	worst
recipe	control	**+0.0000**	**-0.0008**	**-0.0008**	**-0.0008**	**+0.0016**	**+0.0008**	**+0.0016**
recipe-short	control-short	**-0.0008**	**-0.0008**	**+0.0008**	**-0.0008**	**-0.0016**	**-0.0023**	**-0.0023**
t00	control-short	**-0.0094**	**-0.0086**	**-0.0008**	**-0.0094**	**-0.0016**	**-0.0023**	**-0.0094**
t48	control-short	**-0.0070**	**-0.0062**	**-0.0008**	**-0.0047**	**-0.0008**	**-0.0023**	**-0.0070**
t12	control-short	**-0.0023**	**-0.0016**	**+0.0008**	**-0.0031**	**+0.0000**	**-0.0023**	**-0.0031**

The H1 read: each scored condition's seed-mean holdout exact match minus that of the control of the same length, per op. Bold marks a gap within the gate of 0.02; the partial band runs to 0.05.



Holdout exact match per op, against the gate. *One panel per op; conditions down the side, in their own ink. Each dot is the seed mean, the bar the seed range. The grey strip behind a row is ±0.02 around the seed-mean accuracy of the control of the same length, so a dot outside its strip misses the H1 gate; the two control rows carry their own strips for reference.*



Validation loss over training. *Five thin lines per condition on the step axis: the control in grey and the recipe in blue. The ghost lines behind them are the ex-2.1.10 primary (nine seeds) and its control (three), which trained on the one-op grammar at the same step count; their floor sits lower because their corpus held about seventeen copies of each distinct line, where this one holds most lines once. Log scale.*

H1, holds. All 30 condition-op comparisons are within 0.02 of the control of the same length; the largest gap is -0.0094, `t00` on `mix`.

Placement under the recipe reproduces (H2)

H2. The ex-2.1.10 recipe places *red* on the new grammar as it did on the old one. On the `mix` probe lines, with five fresh seeds, all of the ex-2.1.10 placement gates hold. Each statistic is a seed mean unless the criterion says otherwise: containment $\bar{\alpha}$ at op1 at most 0.1; a leading softmin weight of the red group at the embedding of at least 0.4; contrast at least 0.2; grading r^2 no more than 0.1 below the 0.782 the survey measured on this recipe; m_line at least 0.8 of its ex-2.1.10 value of 0.420; and, read per run rather than on the seed mean, retention at least 0.8 of the running peak for every run whose peak reaches 0.2, and no latched run.

Partial: every criterion holds except one of m_line in the 0.6–0.8 band or contrast in the 0.1–0.2 band. Contrary: a latch or a containment miss. That would mean something in the new grammar changed what the pull finds. If instead the margin fell below the partial band while the other criteria held, the placement would be there but weaker, and the H3 selection would lean toward the proposals.

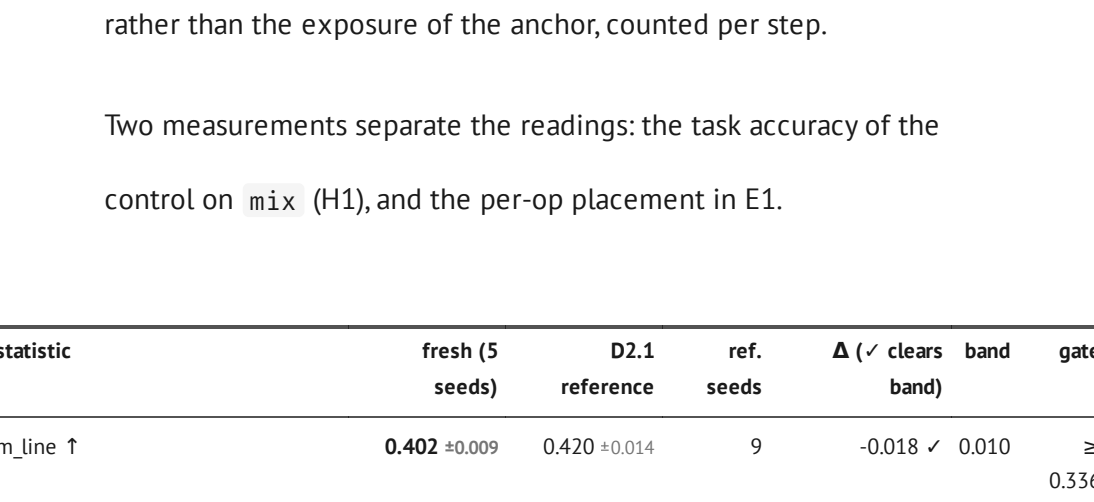
Two things in the new grammar could do that, and we read a miss against both. The first is the op word. The labeller never sees it, and reads the same two operand roles on every op.

The second is exposure. Only a sixth of the corpus is `mix`, and those lines are drawn from every pair rather than from the on-grid ones. So over a run the model sees a given `mix` probe pair about a fiftieth as often as in D2.1, and never sees most of them (see the corpus). What the anchor sees is closer to unchanged. The corpus is the same size, and the share of lines the labeller fires on follows the operand redness: 6.4% of all pairs, against 6.3% of the on-grid ones in D2.1. So the exposure that fell is the exposure of the task, counted per pair, rather than the exposure of the anchor, counted per step.

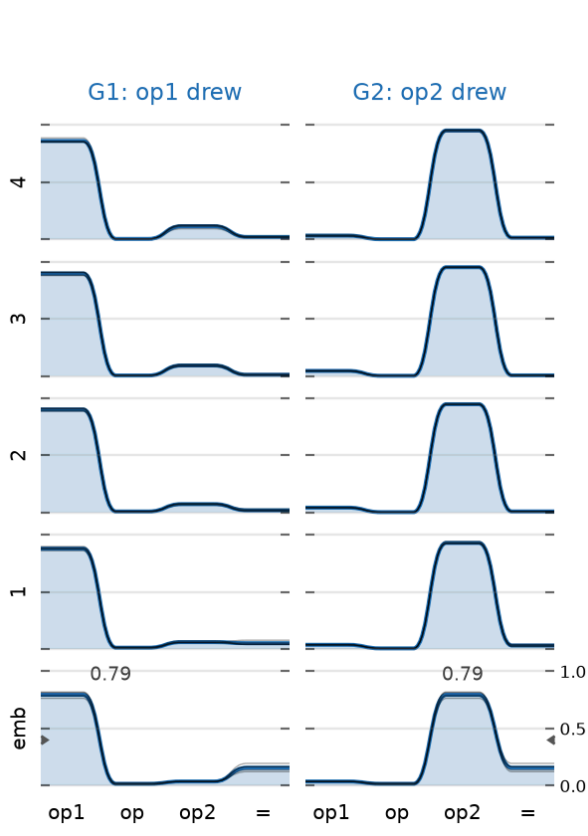
Two measurements separate the readings: the task accuracy of the control on `mix` (H1), and the per-op placement in E1.

statistic	fresh (5 seeds)	D2.1 reference	ref. seeds	Δ (✓ clears band)	band	gate
m_line ↑	**0.402** ±0.009	0.420 ±0.014	9	-0.018 ✓	0.010	≥ 0.336
m_span	0.749 ±0.017	0.738 ±0.032	9	+0.011	–	–
containment ᾱ ↓	**0.062** ±0.020	0.050 ±0.039	9	+0.012 ·	0.024	≤ 0.1
lead weight at emb ↑	**0.79** ±0.03	–	–	–	–	≥ 0.4
contrast ↑	**0.859** ±0.005	0.856	3 (survey)	+0.003 ·	0.008	≥ 0.2
grading r² ↑	**0.819** ±0.036	0.782	3 (survey)	+0.037 ✓	0.035	≥ 0.682
retention, min over runs ↑	**0.994** ±0.005	0.988 ±0.010	9	+0.005 ·	0.007	≥ 0.8
non-red op1 weight, max over runs ↓	**0.17** ±0.04	–	–	–	–	≤ 0.5

The placement statistics of the recipe on the `mix` probe lines, five fresh seeds, beside the D2.1 reference. The reference is the ex-2.1.10 primary's nine seeds where its stored metrics carry the statistic, and the survey's three-seed re-run of the same recipe (its `ref` arm) for contrast and grading, which ex-2.1.10 did not store per run; the lead weight and the latch have no stored reference. Δ is fresh minus reference; a ✓ beside it means the difference clears the band ($2\sigma\sqrt{(1/5 + 1/n)}$ with the frozen per-run σ), a dot that it does not. Bold marks a value inside its gate. Retention and the latch are read per run, so their rows print the worst run.



m_line over training. *The trajectory instrument reads one line per color on the `mix` probe set, every 50 steps. Five thin blue lines are the recipe's fresh seeds; the ghost lines behind them are the nine seeds of the ex-2.1.10 primary on the one-op grammar, on the same step axis. The grey lines are the un-anchored control. The dip over the last tenth is the anchor anneal.*



Per-group softmin profiles for the recipe on the `mix` lines. *Columns are the label groups (G1 weights probe lines by P(only op1 drew), G2 by P(only op2 drew)); rows are residual slices with the embedding at the bottom; the shaded smooth-step is the seed-mean softmin weight over the span roles, with a hairline per seed. On the embedding row, the caret on each column's outer axis marks the lead gate (0.4; uniform is 0.25), and the number above each group's own operand is the seed-mean weight there. Deep-slice op2 contrast +0.86 ±0.00 (gate ≥ 0.2). The layout is ex-2.1.10's, whose primary put 0.77 on the own operand at the embedding and 0.9 or more at depth.*

H2, holds. Every placement criterion holds on the `mix` lines at five fresh seeds: seed-mean m_line 0.402, 0.96 of its ex-2.1.10 value, grading r² 0.819, contrast 0.86, containment 0.062, no latched run.

The plateau from the survey transfers (H3)

H3. The plateau found in the survey behaves to the recipe rather than to the one-op grammar. At least one proposal is feasible at fresh seeds on the new grammar, and its seed-mean m_line on the `mix` probe lines exceeds `recipe-short` (the recipe at the same step count) by more than one band. Feasibility is read as the survey read it, with the constraints listed once in the selection rule below. The grading margin is one of them, and so is a floor on the worst seed: every run's m_line at least 0.6 of its ex-2.1.10 value, since the bands use the frozen per-run σ and a candidate's own spread would otherwise not count against it.

Partial: a proposal is feasible and its m_line sits within a band of `recipe-short`; or a proposal clears every survey constraint and exceeds `recipe-short` by more than a band, but misses the grading margin. Contrary: no proposal is feasible, or every feasible proposal sits more than a band below `recipe-short`. Either would say the plateau was specific to the one-op grammar, and the recipe would carry D2.2. A proposal that misses only the per-run floor is reported as unstable at fresh seeds rather than as weak, with the seed range printed.

The selection rule. Among the candidates (the recipe at both lengths and the three proposals), keep those feasible at fresh seeds. On the seed mean: task within the gate on every op against the control at the same length, containment at least the partial floor, and grading r^2 no more than 0.10 below the full-length recipe's fresh value, clearing that floor by at least one per-run σ. On every run: retention, no latch, and m_line at least the partial margin of its ex-2.1.10 value. Among those, require contrast at the full gate, and take the highest seed-mean m_line on the `mix` probe lines. Every candidate within one m_line band of that highest value ties with it, and the tie goes to the larger grading margin, then to the full-length recipe, then to `recipe-short`. If no other candidate is feasible, or none of the feasible candidates clears the full contrast gate, the full-length recipe is adopted, and if it is not itself feasible the report says which check it missed beside the adoption. Whichever point is adopted, the D2.2 experiments carry it. The Findings line for this section names the adopted point, and it lands in `experiment.py` for the D2.2 experiments that follow.

Winner's curse. The numbers from the survey are proposals. So the table prints the one-op value for each candidate beside the fresh one, with the per-run σ behind the bands and the band a difference has to clear. The survey re-ran the recipe at both lengths, and those runs sit in the same table, since they are the one-op values the fresh recipe arms are read against.

candidate	statistic	survey seeds	fresh (5 seeds)	Δ (+ clears band)	σ per run	band
recipe	m_line	0.420	5	0.402 +0.009	−0.018 ✓	0.0088	0.015
	r2_sim	0.782	5	0.819 +0.056	+0.036 ✓	0.0240	0.035
	contrast	0.806	5	0.859 +0.001	+0.053 ✓	0.0057	0.008
	alpha_op1	0.039	5	0.062 +0.033	+0.023 ✓	0.0212	0.031
	holdout_em	0.996	5	0.999 +0.002	+0.003 ✓	0.0087	0.013
	retention	0.990	5	0.994 +0.001	+0.004 ✓	0.0061	0.009
recipe-short	m_line	0.422	5	0.401 +0.007	−0.020 ✓	0.0088	0.015
	r2_sim	0.806	5	0.875 +0.037	+0.068 ✓	0.0240	0.035
	contrast	0.849	5	0.869 +0.009	+0.020 ✓	0.0057	0.008
	alpha_op1	0.026	5	0.063 +0.025	+0.037 ✓	0.0212	0.031
	holdout_em	1.000	3	0.999 +0.001	−0.001	0.0087	0.013
	retention	0.997	3	0.995 +0.004	−0.002	0.0061	0.009
t00	m_line	0.534	5	0.477 +0.032	−0.057 ✓	0.0088	0.015
	r2_sim	0.690	5	0.814 +0.041	+0.144 ✓	0.0240	0.035
	contrast	0.267	5	0.362 +0.021	+0.094 ✓	0.0057	0.008
	alpha_op1	0.062	5	0.089 +0.024	+0.027 ✓	0.0212	0.027
	holdout_em	0.983	5	0.991 +0.014	+0.008	0.0087	0.011
	retention	0.963	5	0.985 +0.013	+0.009 ✓	0.0061	0.008
t48	m_line	0.527	5	0.465 +0.020	−0.063 ✓	0.0088	0.011
	r2_sim	0.716	5	0.866 +0.025	+0.150 ✓	0.0240	0.030
	contrast	0.740	5	0.856 +0.007	+0.116 ✓	0.0057	0.007
	alpha_op1	0.045	5	0.077 +0.015	+0.032 ✓	0.0212	0.027
	holdout_em	0.994	5	0.995 +0.006	+0.009	0.0087	0.011
	retention	0.976	5	0.986 +0.005	+0.010 ✓	0.0061	0.008
t32	m_line	0.513	5	0.461 +0.015	−0.051 ✓	0.0088	0.011
	r2_sim	0.782	5	0.842 +0.021	+0.060 ✓	0.0240	0.030
	contrast	0.378	5	0.419 +0.014	+0.040 ✓	0.0057	0.007
	alpha_op1	0.022	5	0.039 +0.004	+0.017	0.0212	0.027
	holdout_em	0.984	5	0.991 +0.004	+0.013 ✓	0.0087	0.011
	retention	0.970	5	0.977 +0.015	+0.007	0.0061	0.008

Each candidate's one-op survey numbers beside the fresh ones on the six-op grammar. The survey column is the trial's five-seed mean (for the proposals) or the survey's own three-seed re-run of the recipe at that length; fresh is the seed mean here with half the seed range, Δ is fresh minus survey, with a ✓ where the difference clears the band for those seed counts and a dot where it does not. σ is the frozen per-run spread from ex-2.1.10 behind every band.

Survey against fresh, per candidate. *The same numbers as the table above, drawn: the hollow marker is the survey value, the filled dot the fresh seed mean with a bar for the seed range, and the grey strip the band around the survey value for those seed counts. A dot outside its strip is a difference the resolution rule counts.*

candidate	task	containment	contrast: in partial	contrast: grading	retention	no latch	m_line: feasible floor	contrast: ≥ 0.2	m_line: grading ↑	grading r^2 ↑
recipe	✓	✓	✓	✓	✓	✓	✓	✓	0.402 +0.009	0.819 +0.056
recipe-short	✓	✓	✓	✓	✓	✓	✓	✓	0.401 +0.007	0.875 +0.037
t00	✓	✓	✓	✓	✓	✓	✓	✓	0.477 +0.032	0.814 +0.041
t48	✓	✓	✓	✓	✓	✓	✓	✓	0.465 +0.020	0.866 +0.025
t32	✓	✓	✓	✓	✓	✓	✓	✓	0.461 +0.015	0.842 +0.021

Feasibility, as the selection rule reads it. Seed-mean checks: task within 0.02 of the same-length control on every op, containment ≥ 0.1, contrast ≥ 0.1, grading r^2 at least 0.10 below its floor of 0.719, the full-length recipe's fresh value less 0.1. Per-run checks: retention at 0.8 of the same-length peak for every run whose peak reaches 0.2, no run with more than 0.5 of the non-end group's weight on op1, every run's m_line at least 0.6 of 0.420. The selection then requires contrast at the full gate and takes the highest m_line, ties within one band (0.011) going to the larger grading margin. ✓ marks the adopted point: t candidate(s) within one band (0.011) of the highest feasible m_line (0.477).

m_line against grading r^2 for the five candidates. *Filled dots are single seeds, the ring is the seed mean, and the hollow diamond is the same candidate's one-op survey value. The grey strip is one band (0.011) either side of `recipe-short`'s seed mean, which the H3 verdict reads the proposals against; the dotted line is the grading floor of the selection rule (0.719, before its 3σ margin). The adopted point is labelled ★.*

What grading looks like. *One panel per candidate: each point is one of the 216 grid colors, drawn in its own color, at its similarity-to-red target (sim^1.5) along the bottom and its seed-mean alignment at op1 up the side, averaged over the four post-attention slices. The r² in each title is the squared correlation of those two axes on the seed-mean cloud; the tables report the per-seed r^2 averaged, which is a little lower. A cloud that rises in a line is graded; one that steps from a floor to a ceiling is thresholded.*

H3, holds. A feasible proposal exceeds `recipe-short` (0.401) by more than a band (0.011): `t00` +0.075, `t48` +0.062, `t32` +0.059. Adopted: `t00`.

Post hoc: the selection rule, with the gates it left out

This is a deviation, written after the numbers were read. The selection rule above was frozen with two holes in it, and the point it adopts falls through both.

The first hole is H2's lead gate. Feasibility carries every H2 gate except the lead weight of the red group at the embedding, which H2 gates at 0.4; per candidate that weight is `recipe` 0.79, `recipe-short` 0.85, `t00` 0.53, `t48` 0.46, `t32` 0.48. Add the gate, and the highest feasible m_line is `t48`.

The second hole is H4. The rule ranks on placement and never looks at what the plain projection costs, and on every proposal that cost is most of the non-red `mix` lines: the deficit under `projection` is `recipe` 0.024, `recipe-short` 0.014, `t00` 0.602, `t48` 0.498, `t32` 0.598, against H4's gate of 0.05. The cost sits on the syntax embeddings (E.2.67), and the operand-only edit routes around it, but an operating point that needs a routed edit before its first intervention is a poor base for the anchored-op experiments. Add H4's selectivity gate as well, and only the two recipe lengths remain; at five seeds they sit within a band of each other, and E6's twenty-seed read breaks the tie: at twenty seeds the full recipe leads by 0.012, more than the 0.007 band, so it takes the tie.

The amended rule adopts **`recipe`**. The frozen H5 verdict stands as written, and the proposals keep their reads: they place *red* with a larger margin, and they pay for it on the syntax embeddings. Whether that cost can be removed rather than routed around is the untied-readout question in the Discussion.

D2.2 builds on `recipe-short`, by decision rather than by rule. Both rules rank on margin, and at twenty seeds the full recipe leads on margin by 0.012, about two bands. The short arm is ahead on the rest of what the anchored-op experiments will lean on: it grades better (r^2 0.886 against 0.821), puts more of the pull on the drawing operand at the embedding (0.83 against 0.79), has a little more contrast, gives the same H4 read, and costs half as much to train. Those experiments read the grading and the operand lead more than they read the margin, so the graded point is the better base. The decision was taken on 2026-09-10, after E6 was read, and this paragraph is its record.

Suppression transfers (H4)

H4. Projecting the axis out removes *red* on every op, at the selectivity ex-2.2.1 reached. On the recipe and on the adopted point, under `projection`, seed-mean accuracy on the red lines of each op falls to at most 0.2, and the seed-mean non-red deficit on the `mix` lines is at most 0.05. Partial: exactly one clause holds, or removal holds on all but one op, or the deficit sits in the 0.05–0.1 band.

The deficit gate is the band ex-2.2.1 landed in (0.024, partial at its 0.02 gate), and we report the 0.02 read beside it. That cost came from the syntax embeddings, and E2 reads the embeddings of the six op words.

Contrary on removal: red accuracy near the clean value on one of the new ops. That would say the model reads *red* off the axis on lines of that op, and the per-op alignment in E1 should show it first. Contrary on selectivity: a deficit above 0.1. That would send the intervention-tuning pass to the `operands` and `shaped` rows before anything is anchored.

condition, operator	mix red acc ↓	add red acc ↓	screen red acc ↓	multiply red acc ↓	lighten red acc ↓	darken red acc ↓	ex-2.2.1 (mix)
recipe, projection	**0.06** ±0.05	0.32 ±0.20	0.23 ±0.10	**0.17** ±0.12	0.23 ±0.09	0.24 ±0.23	0.09 ±0.10
recipe, operands	0.09 ±0.06	0.35 ±0.21	0.24 ±0.11	0.19 ±0.12	0.25 ±0.09	0.32 ±0.26	0.13 ±0.13
recipe, shaped	0.42 ±0.28	0.66 ±0.28	0.58 ±0.25	0.51 ±0.29	0.52 ±0.21	0.59 ±0.36	0.60 ±0.29
recipe, ablate	0.08 ±0.06	0.35 ±0.19	0.26 ±0.10	0.20 ±0.13	0.27 ±0.10	0.33 ±0.27	0.12 ±0.12
recipe, clean	1.00 ±0.01	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00

Removal on the red lines of each op, for `recipe`. Each value is seed-mean exact-match accuracy on the red lines (dose ≥ 0.8) under the operator, with half the seed range; the last column is the same operator on ex-2.2.1's one-op grammar (nine seeds of the D2.1 recipe). Bold marks a `projection` value inside the removal gate, at or below 0.2. The ride-along rows carry no gate. The last row is the clean pass.

condition, operator	mix red acc ↓	add red acc ↓	screen red acc ↓	multiply red acc ↓	lighten red acc ↓	darken red acc ↓	ex-2.2.1 (mix)
t00, projection	**0.00** ±0.00	**0.05** ±0.03	**0.02** ±0.01	**0.01** ±0.01	**0.06** ±0.04	**0.03** ±0.02	0.09 ±0.10
t00, operands	0.08 ±0.09	0.23 ±0.12	0.22 ±0.13	0.20 ±0.14	0.32 ±0.17	0.15 ±0.10	0.13 ±0.13
t00, shaped	0.00 ±0.00	0.07 ±0.05	0.04 ±0.03	0.02 ±0.03	0.09 ±0.05	0.04 ±0.04	0.60 ±0.29
t00, ablate	0.00 ±0.00	0.03 ±0.03	0.02 ±0.02	0.01 ±0.01	0.05 ±0.04	0.02 ±0.02	0.12 ±0.12
t00, clean	0.89 ±0.05	0.92 ±0.04	0.96 ±0.03	0.95 ±0.02	0.95 ±0.02	0.97 ±0.01	1.00 ±0.00

Removal on the red lines of each op, for `t00`. Each value is seed-mean exact-match accuracy on the red lines (dose ≥ 0.8) under the operator, with half the seed range; the last column is the same operator on ex-2.2.1's one-op grammar (nine seeds of the D2.1 recipe). Bold marks a `projection` value inside the removal gate, at or below 0.2. The ride-along rows carry no gate. The last row is the clean pass.

condition, operator	mix non-red deficit ↓	add non-red deficit ↓	screen non-red deficit ↓	multiply non-red deficit ↓	lighten non-red deficit ↓	darken non-red deficit ↓	ex-2.2.1 (mix)
recipe, projection	**0.024** ±0.030	0.001 ±0.002	0.003 ±0.005	0.006 ±0.008	0.000 ±0.002	0.001 ±0.002	0.024 ±0.063
recipe, operands	-0.000 ±0.001	-0.000 ±0.000	-0.000 ±0.001	0.000 ±0.002	-0.000 ±0.001	0.000 ±0.000	0.000 ±0.001
recipe, shaped	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000	0.000 ±0.000
recipe, ablate	0.008 ±0.009	0.001 ±0.001	0.001 ±0.002	0.002 ±0.004	0.001 ±0.002	0.001 ±0.001	0.012 ±0.021

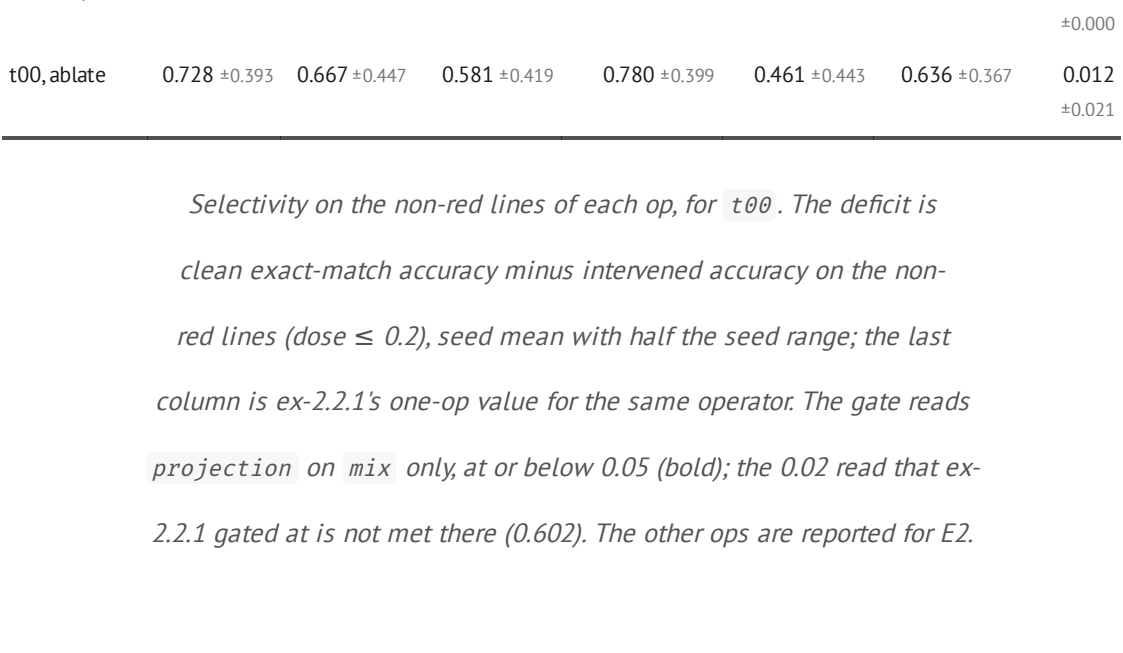
Selectivity on the non-red lines of each op, for `recipe`. The deficit is clean exact-match accuracy minus intervened accuracy on the non-red lines (dose ≤ 0.2), seed mean with half the seed range; the last column is ex-2.2.1's one-op value for the same operator. The gate reads `projection` on `mix` only, at or below 0.05 (bold); the 0.02 read that ex-2.2.1 gated at is not met there (0.024). The other ops are reported for E2.

condition, operator	mix non-red deficit ↓	add non-red deficit ↓	screen non-red deficit ↓	multiply non-red deficit ↓	lighten non-red deficit ↓	darken non-red deficit ↓	ex-2.2.1 (mix)
t00, projection	0.602 ±0.367	0.421 ±0.246	0.401 ±0.299	0.532 ±0.397	0.388 ±0.367	0.442 ±0.357	0.024 ±0.063
t00, operands	0.017 ±0.017	0.007 ±0.012	0.003 ±0.005	0.003 ±0.003	0.001 ±0.003	-0.001 ±0.003	0.000 ±0.001
t00, shaped	0.732 ±0.255	0.518 ±0.199	0.523 ±0.222	0.638 ±0.297	0.380 ±0.321	0.481 ±0.396	0.000 ±0.000
t00, ablate	0.728 ±0.393	0.667 ±0.447	0.581 ±0.419	0.780 ±0.399	0.461 ±0.443	0.636 ±0.367	0.012 ±0.021

Selectivity on the non-red lines of each op, for `t00`. The deficit is clean exact-match accuracy minus intervened accuracy on the non-red lines (dose ≤ 0.2), seed mean with half the seed range; the last column is ex-2.2.1's one-op value for the same operator. The gate reads `projection` on `mix` only, at or below 0.05 (bold); the 0.02 read that ex-2.2.1 gated at is not met there (0.602). The other ops are reported for E2.

condition, distance, pass	mix ↓	add ↓	screen ↓	multiply ↓	lighten ↓	darken ↓
recipe, decoded answer, projection	0.31 ±0.06	0.31 ±0.10	0.32 ±0.08	0.36 ±0.11	0.37 ±0.10	0.43 ±0.18
recipe, decoded answer, clean	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00
recipe, expected, projection	0.32 ±0.06	0.31 ±0.10	0.32 ±0.07	0.36 ±0.11	0.37 ±0.09	0.43 ±0.17
recipe, expected, clean	0.01 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00	0.00 ±0.00
t00, decoded answer, projection	0.48 ±0.08	0.55 ±0.05	0.50 ±0.06	0.60 ±0.09	0.49 ±0.08	0.65 ±0.06
t00, decoded answer, clean	0.02 ±0.01	0.01 ±0.01	0.01 ±0.00	0.01 ±0.01	0.01 ±0.00	0.00 ±0.00
t00, expected, projection	0.48 ±0.08	0.56 ±0.04	0.51 ±0.06	0.61 ±0.10	0.50 ±0.07	0.66 ±0.07
t00, expected, clean	0.06 ±0.01	0.07 ±0.02	0.05 ±0.01	0.04 ±0.02	0.05 ±0.02	0.04 ±0.01

Answer distance on the red lines under `projection`, in the unit cube. The decoded row is the distance from the argmax answer to the true one; the expected row takes the expectation of that distance under the whole answer distribution over the color vocabulary. Each value is the seed mean with half the seed range; the clean pass sits under each projected row. On ex-2.2.1's one-op grammar the decoded distance under the projection was 0.29 ±0.06; a one-level step on this grid is 0.20.



The bound and the write, per site and per op, for the recipe under `projection`, signed. *For each op's probe lines, at each (slice, position), seed mean. The shaded band runs from the 1st to the 99th percentile of the signed clean alignment over the non-red lines; the scorer's bound is the arcsine of the 99th-percentile |α|, which is at least the further edge and equal to it where one tail holds the extreme, so this is close to the map the earlier version drew folded onto one side. The dashed pair is the 99th-percentile |α| arriving at the operator, whose arcsine is the write, mirrored about zero because the scorer keeps only its magnitude. The embedding row matches the clean band by construction. One row per slice with the embedding at the bottom, one column per op, and every panel on the same scale, so a write that pokes above its bound reads as a dashed line clearing the further band edge. Sites where the seed-mean write exceeds the seed-mean bound, per op: `mix` 3, `add` 3, `screen` 4, `multiply` 3, `lighten` 5, `darken` 4 of 24 post-embedding sites.*

H4, contrary. `recipe`: red accuracy under projection 0.06–0.32 across the ops, `mix` non-red deficit 0.024; `t00`: red accuracy under projection 0.00–0.06 across the ops, `mix` non-red deficit 0.602. Removal misses on `add, screen, lighten, darken` for `recipe`. The `mix` deficit on `t00` is above 0.1.

Exploratory analyses

Perceptions of vulnerability to pain.

3.1 – pain by subtraction

Every PC starts each at the pickle line in each of each up rather than at the. And. Into the into the. Pickle the pickle pickle pickle pickle pickle pickle up up up pickle pickle pickle pickle pickle pickle

Discussion

Here is what we think happened, in plain terms.

The anchoring recipe does not care about the grammar. We trained on six rules instead of one, with the labeller still only ever looking at the two operands, and *red* landed on the axis the same way it did in D2.1: no task cost on any op (H1), every placement gate met (H2), and the same numbers on the lines of every op (E1). The recipe needs its own stream of labelled operands each step, the corpus kept that, and the rest of the line can do whatever it likes. So the "recipe is grammar-specific" row of the D2.2 risk table closes.

The survey's proposals are real, and smaller than advertised. All three came in about 0.05 of m_line below their survey values, the recipe about 0.02 below its own. That is the winner's curse we priced in: when you pick the best of many noisy trials you tend to pick one that got lucky, so a fresh run scores lower.[1] About two thirds of the advantage over `recipe-short` survived, the proposals grade better than the survey said (the other side of the same luck), and they spread more across seeds than the recipe does. E5 says the H3 differences hold up under the wider band too.

The selection rule picked `t00`, and we are not carrying it. The rule ranked on margin and checked most of the H2 gates, and `t00` won. But it misses the one gate the rule left out, the lead weight at the embedding (0.33 against 0.4), and its contrast is 0.36 where the recipe's is 0.86. More to the point, it costs something the rule never looked at. At five times the recipe's anchor weight, the axis ends up on the `=` and op-word embeddings of the shared embedding table (E2: `=` at 0.93 against 0.40), and projecting the axis out of every position then breaks 0.6 of the non-red `mix` lines, with some seeds losing nearly all of them. The same is true of `t48` and `t12` (E7). Editing only the operand positions gets around it (0.017 on `mix`), and the prereg's contrary clause already sends the intervention-tuning pass to those tokens. But an operating point that needs a routed edit before its first intervention is a weak base for the anchored-op experiments, so the post hoc read adds the two missing gates and adopts the recipe, and D2.2 builds on its short arm. Whether the syntax embeddings can be made clean rather than routed around is the tied-readout question, and it wants settling before the D2.3 swap, which rotates every position.

Full length or half? We ran fifteen more seeds of each recipe length to answer this properly (E6). At twenty seeds the full recipe keeps a 0.012 margin lead, about twice the band, and the short one grades better (r^2 0.89 against 0.82) and leads better at the embedding (0.83 against 0.79). Neither is wrong. The post hoc rule ranks on margin, so it names the full recipe; we chose the short arm anyway, and the choice is recorded in the post hoc section. The anchored-op experiments lean on grading and on the operand lead more than on margin, the H4 read is the same, and the short arm is half the compute.

Removal looked partial on four ops, and mostly is not. H4's removal statistic is whether the model still gets the answer right once the axis is projected out. On the recipe that held on `mix` (0.06 red-line accuracy) and missed on four of the new ops (0.17 to 0.32). The reason is the ops, not the anchor. On `add`, `screen`, `multiply`, `lighten`, and `darken`, most red lines have an answer that would be the same if the red operand were a little less red, because the op saturates or picks the other operand's channel; a model that has lost *red* can still answer those. E8 splits each op's red lines by whether the answer depends on the red operand's redness at all, and on the dependent lines the projection removes *red* on every op (accuracy 0.00 to 0.18 on the recipe, 0.02 or less on `t00`). The misses in the H4 table are the lines where the correct answer never needed *red*. For the next grammar we want both kinds: some ops whose answers depend on both operands and spread through the cube, and some anisotropic ones like these, whose answers sit on a face or an edge, so the E8 split has both to read; that is the diverse-op item, and it should be checked with the answer clouds and the E8 split before anything is anchored on it.

Selectivity on the recipe transfers. The `mix` non-red deficit under projection is the 0.024 from ex-2.2.1 at five seeds and 0.026 at twenty, and the other ops lose nothing to speak of. The write stays inside the bound at most sites (the stacked figure under H4), and where it pokes above, it is at the op word, which is the one new token.

The blind span behaves. On the lines whose answer is redder than either operand, the answer's own redness shows on the axis at `=` and at the answer position, by 0.15 at most, and the projection leaves most of those lines intact (E3). The anchor never sees the answer, so that redness is computed off the axis, as the ex-2.1.10 scope note said it would be. `multiply` has the most such lines and the largest shift, so it is where a labeller that also reads the answer would show up first; that labeller is filed under the span variants.

More rules did not make a straighter cube, on this probe. At six ops the operand's RGB is less linearly decodable at its own slot than at three (0.52 against 0.85 to 0.87), in both matchings, though the six-op seeds spread widely (±0.14) and the one-op arm of the `per-op` matching is in a memorization regime, so the read is a trend over three points rather than a result. The answer at `=` gives no clean read either way. Whether the model builds op-specific readings of its operands is a probe question for the anchored-op experiments, which read those sites anyway (E4).

Which op to anchor next. The relevance table favors `mix` (94% of its lines have an answer no other op gives) and, among the new ops, `multiply`. E3 adds that `multiply` is where the answer's redness shows on the axis most, and E8 that its red lines are about half dependent. The suppression prereg can choose on what it wants to show: `mix` for the sharpest test, `multiply` for a rule the anchor has never seen.

Method

Two things change for the model against D2.1: the corpus carriers six rules at the same total size, and a token between the operands decides the answer.[1] The dependency named in the design lands before the DAG, in `sca_data`, and the table below is its specification.

The grammar

Six ops on the `XISG` grid (six levels per channel: 0, 5.4, 9.3, 11.5), with every color one token and every op one token. Each op is computed per-channel on its 3.5 k-site and snapped to the nearest grid level. So every op answers every pair with a color in the vocabulary, and a line can be written for any pair. Where the rule lands on the grid by itself, no rounding happens, and `min` on those pairs is the D2.1 op.

Across the pairs, the snap rounds up about as often as down. `screen` and `multiply` never land halfway between two levels, so their signed rounding error is zero. `min` lands halfway on half of its channel sums, and those ties go to the even level (0, 4, or 12), which tends ten of the eighteen cases down and eight up.[2]

Each op has 46,656 lines, of which 2,975 are red and 13,225 non-red by dose, the larger of the two operand rednesses. Those counts are the same for every op, since dose reads the operands.

op	per channel	on grid	redder than both
`min`	$\lfloor x \times y / J \rfloor$	12.9%	4,939 (10.5%)
`add`	$\min(y + s, 11)$	100.0%	2,161 (4.6%)
`screen`	$11 - \lfloor (5 - x)(5 - y) / 11 \rfloor$	17.1%	1,417 (3.1%)
`multiply`	$x \times y / 11$	17.1%	1,891 (8.3%)
`lighten`	$\max(x, y)$	100.0%	0 (0.0%)
`darken`	$\min(x, y)$	100.0%	4,258 (9.1%)

The five named rules are the blend modes of the same names in Photoshop and Krita, on the 0..11 scale; `add` is what Photoshop calls *linear dodge*. `min` is a normal blend at half opacity, which is the per-channel mean.

All six ops are commutative, as `min` was in D2.1, so operand order carries no information and the two operand roles stay interchangeable.

The set bears light. `add`, `screen`, and `lighten` can only raise a channel, `multiply` and `darken` can only lower one; `min` sits between by operands. A `subtract` op would balance `add`, but it would also be the first op whose operand order mattered, so the set stays as it is.

Agreement. How often each pair of ops gives the same answer. On a line where two ops agree, reading the op word gains nothing:

	`mix`	`add`	`screen`	`multiply`	`lighten`	`darken`
`mix`	–	0.0%	0.0%	0.1%	2.6%	4.1%
`add`	0.0%	–	17.6%	0.0%	17.1%	0.0%
`screen`	17.6%	17.6%	–	0.0%	17.1%	0.0%
`multiply`	0.1%	0.0%	0.0%	–	0.0%	17.1%
`lighten`	2.6%	17.1%	17.1%	0.0%	–	0.9%
`darken`	4.1%	0.0%	0.0%	17.1%	0.9%	–

`screen` agrees with `add` on about a third of pairs and with `lighten` on about a third as well, at the light end of the range where all three saturate; `multiply` agrees with `darken` at the dark end. `add` and `lighten` agree on a sixth of pairs. Every other pair agrees on a few percent at most, at the ends of the range.

Op-reference. Take the lines of one op and ask of each line: how many of the other five ops would have given the same answer for this pair? Call that count k. Where k = 0, no other op matches, so the model has to read the op word to get the line right. Where k is larger, the op word narrows the six ops to k + 1, all of which give the same answer, so the model could answer the line without reading it.

The table gives, for each op we might anchor, the share of its lines at each k. The suppression experiments rest on this: when the op concept is removed, only the lines at k = 0 can show it, so an op with most of its lines there makes a sharper test. Which op we anchor stays open until the suppression prereg.

anchored op	0	1	2
`mix`	94.2%	4.5%	1.3%
`add`	62.4%	20.5%	17.1%
`screen`	41.9%	41.0%	17.1%
`multiply`	62.5%	37.0%	0.4%
`lighten`	59.9%	22.1%	17.9%
`darken`	58.8%	39.8%	1.3%

The corpus

100,000 lines, the same count as D2.1. Ops are drawn uniformly, with attention as op the training pairs are drawn uniformly with random operand order; the way ex 2.1.10 drew the `mix` pairs. For each op, a fifth of its distinct pairs is held out, keyed on (op, pair), so a pair held out under `add` may still be trained under `mix`. Corpus seed 0.

D2.1 drew its 100,000 lines from 5,852 distinct lines, so the corpus held about seventeen copies of each there. The 100,000 lines come from six times 46,656 distinct lines, so most distinct lines appear at most once, and most pairs of an op never appear under that op. In 64, the corpus arms keep the 100,000 lines and narrow the op set, while the per-op arms narrow the op set and keep 16,666 lines per op.

The operands are the whole grid for every op, as they were in D2.1, what differs by op is where the answers land. `add` with k = 0 lands 62% of its points white, `screen` crowds the light half of the color and `multiply` the dark half and `mix`, `lighten`, and `darken` spread their answers through it.



Where each op's answers land. One mark per grid color, with area proportional to the number of unordered pairs whose answer is that color, on the scale for all six panels: a mark that fills its grid cell stands for 600 pairs, out of 21,456.

Training

D64.1.4 nGPT with the ex 2.1.5 data config. A step is a batch of 64 random crops of 64 tokens (about ten lines each) from the tokenized corpus. An epoch is enough batches to sample a quarter of the training tokens, which the splits work with an oversample of 16 over the batch size. At the corpus size of D2.1 that comes to 55 steps per epoch, as it did there.

Peak LR is 0.01. Anchoring uses the D2.0 code path: the pooled either-operand labelset (each operand draws at redness^2 + 0.04), the anchor on its at-every-slot, and the anti-subspace term.

Conditions are matched on compute, at the step count of D2.1, rather than on samples per op: by per-op-entry regime of the anchor matches D2.1, and how often the task sees each pair becomes a covariate that H1 reads.

The recipe runs for the 100 epochs of ex 2.1.10, the proposals and its short arm run for the 50 of the survey. The `par-op` arms of E4 keep the step count and shrink the corpus, so they get more and shorter epochs; the conditions table prints them.

Every schedule beyond/arm is a fraction of training, as ex 2.1.11 restated them. LR and anchor warm up over the first 10%, the anchor anneal for the recipe over the last 10% down to a 0.1 floor, and the anti-subspace weight annealing from its peak ratio to 0.3 of the anchor weight by its own end fraction. The parameters for the proposals are the sampled values from the survey, unrounded, in `experiment.py`.

The probe set

The probe set is a fixed list of equations that every run is scored on, separately from training: 27 lines per color as spit, per op for `mix` these are its on-grid partners, so the `mix` probe set is the D2.1 one with the op word changed. For the other ops we draw 27 partners per color once with seed 0, and share them across every run.

Each op then has 5,832 probe lines, test and non-test by dose. We keep the full `darken`,[3] so the probe positions in `sca.compute_evaluation` still read correctly.

Measurements

The placement statistics are the ones from ex 2.1.10, computed by its code on the probe lines of each op from one clean pass per run: m_line and the softmin profiles, containment, the lead-weight, contrast, grading against the one's 5.1 target, and the late-% fraction. Trajectory records m_line every 50 steps. Holdout exact-match is read on a held-out set per op, of the size that ex 2.1.10 used.

The interventions run one bracket-forced pass per run per operator over the six probe sets, through the eval contract, giving the log-softmax at `=` and the write per_line, positional as in ex 2.2.1. Red-line accuracy and the non-red deficit are read per op.

Answer distance. Exact match scores a one-step miss and a far miss the same way. Under suggestion ex 2.2.1 found that most misses are red lines were one step neighbors of the true answer. So beside each accuracy we read the distance in the unit cube from the decoded answer to the true one, as ex 2.2.1 read it on the red lines.

A second version takes the expected value of that distance under the answer distribution, using the whole log-softmax rather than only its argmax. Both are computed per line and averaged per group, for every operator row, and neither is gated.

Noise flaw. The bands use the per-run sd of each statistic at the reference recipe, from the nine seeds of ex 2.1.10, printed in the H1 table; 51 re-measures there here. Gates score seed means against fixed thresholds and do not use the bands. Every difference between two conditions is quoted with its band, and a difference inside the band is reported as not resolved.

Budget

77 training runs: 30 at the full length of the recipe, 25 at half of it, and 12 for the richer op arms at the full length as well, each at a plain D2.1 step count on an L4. Scoring is one clean pass plus four operator passes over six probe sets per run, taking CPU seconds each, and the cube probes of E4 are single 216-line rows. That is well under the cost of ex 2.2.2, which trained 24 runs at twice the step cost. The run took 41 minutes of wall-clock on Modal at twelve containers, 15 of them training, and cost $2.55, with no task failed or retried.

Before the freeze

Before the freeze we ran a calibration: one seed of each control arm on the six-op corpus. It checks that the grammar is learned at both lengths, which means holdout exact match per op near 1, the level the D2.1 control reached. The rounded ops are the ones to watch: if the grammar is not learned at one length or the other, we change the corpus size, the step count, or the set of arms before the freeze, and record the change with a `NOTE` note. Nothing anchored runs before the freeze.

The calibration ran. One seed of each control arm, printed from `reports/mlv-2.2.3/calibration.`, the table below reads it. Both lengths learn all six ops: holdout exact match is 1.0 on every op but one at each length, and that one misses a single line of its 216. The rounded ops are as clean as the others, and held-out surprisal is below 0.02 nats everywhere. So the corpus, the step counts, and the set of arms stay as designed, and `control-short` stays too, since its numbers say it will read the proposals' task cost at their own length without a training deficit of its own. Nothing about the arms or the gates changed after this read.

run	`mix`	`add`	`screen`	`multiply`	`lighten`	`darken`
`control-sd`	0.996	1.000	1.000	1.000	1.000	1.000
`control-short-sd`	1.000	1.000	0.996	1.000	1.000	1.000

▶ Glossary…

1. The winner's curse: when you pick the top result out of many noisy trials, you tend to pick one whose noise ran in its favor, so a fresh run of it usually scores lower. ↩

2. Whereas previously, the operation was constant (+1), so the model had no reason to use that position for anything other than spare compute. ↩

3. If the snap always broke ties the same way every rounded `min` answer would shift in that direction, and the model would learn the shift as part of the rule. The `min` of D2.1 rounded half up, which on the grid would be a whole level on every off-grid pair (D2.1 never trained on those pairs, so the change costs nothing), and the probe set is the on-grid pairs, which are untouched. ↩

4. An info specializer sits between its operands, as in `it mix v2`, rather than before or after them. The frame is the fixed six-token layout of a line: `op1, the op word, op2, =, the answer, and the rewrite at` at the same positions in every line, so the probe code can read a role by its position. ↩